

Uniformly Distributed CORDIC

Mario Garrido¹, Senior Member, IEEE, Daniel Medina, Pedro Paz²,
and Marisa López-Vallejo³, Senior Member, IEEE

Abstract—This paper presents the uniformly distributed (UD) coordinate rotation digital computer (CORDIC). It receives this name because the distance between its rotation angles is uniform. The paper presents two versions of the UD CORDIC: the binary UD CORDIC and the canonical signed digit (CSD) UD CORDIC. Both simplify the control part of the CORDIC by obtaining the micro-rotation directions straightforwardly from the binary representation of the rotation angle, with the particularity that, in the CSD UD CORDIC, the binary representation is transformed first into a CSD number. Compared to previous CORDIC approaches that simplify the control part, the proposed rotators further simplify the micro-rotation stages. This is achieved by using more efficient angles for the micro-rotations in the binary UD CORDIC and by merging consecutive micro-rotation stages in the CSD UD CORDIC. Thus, the simplification of the control part of the rotators, the careful selection of the micro-rotation angles, and their optimized shift-and-add implementation results in CORDIC architectures that require the smallest number of adders among pipelined CORDIC implementations for a given resolution so far. Additionally, experimental results of the proposed approach on field-programmable gate array (FPGA) and application-specific integrated circuit (ASIC) are provided, showing improvements with respect to previous approaches in multiple figures of merit, mostly area, clock frequency, and power consumption.

Index Terms—Coordinate rotation digital computer (CORDIC), uniformly distributed (UD) angles, micro-rotation, canonical signed digit (CSD), digital circuit, pipelined.

I. INTRODUCTION

THE coordinate rotation digital computer (CORDIC) [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30] is a well-known algorithm to calculate rotations in hardware. Fig. 1 shows the basic structure of the CORDIC algorithm. The input $x + jy$ is rotated by an input angle θ by means of a series of M micro-rotations by the angles α_i , $i = 0, \dots, M-1$, where the sum of the angles α_i is equal to θ . This allows for selecting angles α_i that are rotated easily by shift-and-add operations, which results in resource-efficient implementations.

The original CORDIC algorithm [1] relies on angles of the form $\alpha_i = \tan^{-1}(2^{-i})$, being i a natural number. The

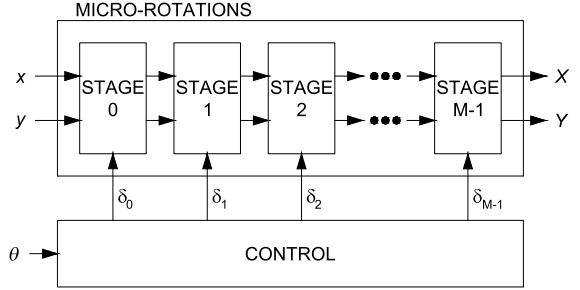


Fig. 1. Basic architecture of the CORDIC algorithm.

micro-rotations by these angles can be calculated with only two adders and shift operations. In fact, in pipelined implementations of the CORDIC algorithm, the shifts are constant and simply consist of hard-wired connections, which do not have any hardware cost.

Through the years, many improvements to the original CORDIC have been proposed. One such improvement is to combine several micro-rotation stages into a single stage [2], [3], which reduces the latency of the CORDIC. Another alternative is to skip and/or repeat micro-rotations [4]. In other approaches, the micro-rotations are represented by Taylor series approximations [5], [6], [7], [8] or are divided into a coarse and a fine part [9], [10]. Other works add more angles to each micro-rotation stage, as in the redundant CORDIC [11]. Furthermore, some works focus on reducing the rotation memory [9], [13], [14] or compensating the scaling factor of the CORDIC [5], [6]. And other solutions propose new sets of micro-rotations with faster convergence [15], [16]. Finally, reviews of CORDIC techniques are available in [17], [18], and [19].

As can be observed, there is a large variety of approaches that deal with different topics related to the CORDIC rotator. Although most of them target the micro-rotation stages, some of them focus on simplifying the control block, which provides the control signals δ_i for the CORDIC stages, being the exact rotation angles at each micro-rotation stage obtained as $\delta_i \alpha_i$.

For implementing the control block, there are three main approaches. The first alternative is to use a memory [9], [20] where the δ_i values are stored. A second approach is to calculate the remaining angle [1], [15]. This is the most common approach in the literature. It compares the remaining rotation angle at each stage to one or several thresholds to determine the δ_i values. The third alternative is to obtain all the δ_i values from θ directly instead of calculating them from the remaining angles [8], [14], [21], [22], [23], [24]. This idea relies on the fact that some consecutive micro-rotation

Received 6 October 2024; revised 20 January 2025 and 20 March 2025; accepted 7 April 2025. Date of publication 22 April 2025; date of current version 27 October 2025. This work was supported in part by MCIN/AEI/10.13039/501100011033 and “ERDF A way of making Europe” under Project PID2021-126991NA-I00 and in part by MCIN/AEI/10.13039/501100011033 and “ESF Investing in your future” under Grant RYC2018-025384-I. This article was recommended by Associate Editor S. Perri. (Corresponding author: Mario Garrido.)

The authors are with the Department of Electronic Engineering, ETSI de Telecomunicación, Universidad Politécnica de Madrid, 28040 Madrid, Spain (e-mail: mario.garrido@upm.es; daniel.medina.merino@alumnos.upm.es; p.pazm@alumnos.upm.es; m.lopez.vallejo@upm.es).

Digital Object Identifier 10.1109/TCSI.2025.3560802

angles fulfill $\alpha_i/\alpha_{i+1} \approx 2$. Thus, by normalizing the rotation angle to the smallest micro-rotation angle, the δ_i values are determined from the binary representation of the rotation angle straightforwardly.

Although the relation $\alpha_i/\alpha_{i+1} \approx 2$ is true for the smallest angles of the conventional CORDIC, it is not true for the first micro-rotations. Therefore, a further advancement in this line has been to consider the angle set $\alpha_i = 2^{-i}$ [8], [24], for which the angle of any micro-rotation is exactly half the angle of the previous one. This reduces the control part of the CORDIC algorithm significantly. However, the fact that the angles of this new angle set are not those in the conventional CORDIC anymore requires the design of new micro-rotation stages that may not use few shift-and-add operations.

In this work, we present the uniformly distributed (UD) CORDIC. It considers the angle set $\alpha_i = 2^{-i}$, which creates a uniform separation of the angles in the circumference. In this paper, we present two different rotators: the binary UD CORDIC and the canonical signed digit (CSD) UD CORDIC. The binary UD CORDIC obtains the values δ_i directly from the binary representation of θ with simple logic operations. The CSD UD CORDIC combines consecutive stages into a single one by taking advantage of the CSD encoding of the rotation angle. The key difference between the proposed rotators and previous approaches that use the angle set $\alpha_i = 2^{-i}$ lies in the δ_i values. Whereas $\delta_i \in \{-1, 0, 1\}$ in [8] and $\delta \in \{0, 1\}$ in [24], the binary UD CORDIC uses $\delta_i \in \{-1, 1\}$. This removes the rotation by 0° , which simplifies the complexity of the micro-rotation stages. For the CSD UD CORDIC, $\delta_i \in \{-2, -1, 0, 1, 2\}$, which is the result of combining consecutive micro-rotation stages. As a result of removing the control part and combining consecutive stages, the CSD UD CORDIC has the outstanding capacity to increase the resolution by a factor of 4 by adding only 2 adders. This capacity has not been observed in CORDIC rotators so far.

The main contributions of this work are:

- The binary and CSD UD CORDIC rotators are proposed.
- In the binary UD CORDIC, the use of $\alpha_i = 2^{-i}$ combined with $\delta_i \in \{-1, 1\}$ simplifies the micro-rotation stages with respect to previous approaches that use $\alpha_i = 2^{-i}$ angles thanks to removing the rotation by 0° .
- The CSD UD CORDIC combines consecutive micro-rotation stages and has the capacity to increase the resolution by a factor of 4 by adding only 2 adders, being the first rotator to achieve this so far.
- The shift-and-add implementation of the micro-rotation stages has been highly optimized in both alternatives.
- In both alternatives, the control part is simplified thanks to the use of the angle set $\alpha_i = 2^{-i}$.
- As a result of all the previous optimizations, the proposed UD CORDIC architectures require the smallest number of adders among pipelined CORDIC implementations so far for any given resolution.
- Hardware architectures are provided for both alternatives, and experimental results on field-programmable gate array (FPGA) and application-specific integrated circuit (ASIC) have been obtained, showing improvements with

respect to previous works in multiple figures of merit, mostly area, clock frequency, and power consumption.

This paper is organized as follows. In Section II, we summarize key concepts that are necessary for understanding the proposed approach. In Section III, we discuss the motivation of the proposed approach in the context of the state-of-the-art. In Section IV, we discuss the methodology that we have followed in our approach. In Sections V and VI, we present the binary UD CORDIC and the CSD UD CORDIC, respectively. In Section VII, we show the experimental results and compare our designs to previous ones. Finally, in Section VIII, we provide the main conclusions of the paper.

II. BACKGROUND

A. Rotations in a Digital System

A rotation in a digital system can be defined as a multiplication by a complex coefficient $C + jS$ according to [31]

$$\begin{aligned} C &= R \cdot (\cos \alpha + \varepsilon_c), \\ S &= R \cdot (\sin \alpha + \varepsilon_s), \end{aligned} \quad (1)$$

where ε_c and ε_s are the quantization errors of the cosine and sine, respectively, and R is the scaling factor. The rotation error is then calculated as $\varepsilon = \sqrt{\varepsilon_c^2 + \varepsilon_s^2}$ and the effective word length of the rotation is [31]

$$WL_E = -\log_2 \varepsilon + \frac{3}{2}, \quad (2)$$

which is defined as the number of bits of the output that are guaranteed to be accurate.

B. The CORDIC Algorithm

The conventional CORDIC algorithm [1] considers the structure in Fig. 1 and uses the set of angles

$$\alpha_i = \tan^{-1}(2^{-i}), \quad (3)$$

where $i = 0, \dots, M - 1$. These angles are rotated either clockwise or counterclockwise, where $\delta_i \in \{-1, 1\}$ indicates the direction of the micro-rotation. According to this, each micro-rotation is calculated as $x_{i+1} + jy_{i+1} = (x_i + jy_i) \cdot (C_i + jS_i)$, where $C_i + jS_i = 2^i + j\delta_i$ is the coefficient that corresponds to the angle α_i . Therefore, the micro-rotations are computed as

$$\begin{aligned} x_{i+1} &= 2^i x_i - \delta_i y_i, \\ y_{i+1} &= 2^i y_i + \delta_i x_i, \end{aligned} \quad (4)$$

where δ_i are obtained from the rotation angle $z_0 = \theta$ as

$$\begin{aligned} \delta_i &= \text{sign}(z_i), \\ z_{i+1} &= z_i - \delta_i \cdot \alpha_i, \end{aligned} \quad (5)$$

where $\text{sign}(\eta) = 1$ if $\eta \geq 0$, and $\text{sign}(\eta) = -1$ if $\eta < 0$. The calculation of δ_i is carried out in the control part of Fig. 1.

Each micro-rotation scales the magnitude of the input by a value $R_i = \sqrt{2^{2i} + 1}$, being the total scaling of the CORDIC the product of the scaling factors at all the stages.

TABLE I
BINARY TO CSD CONVERSION TRUTH TABLE [32]

p_i	b_{i+1}	b_i	b_{i-1}	x_i	$x_{i,s}$	$x_{i,m}$	p_{i+1}
0	0	0	0	0	0	0	0
	0	0	1	1	0	1	1
	0	1	0	1	0	1	1
	0	1	1	0	0	0	0
	1	0	0	0	0	0	0
	1	0	1	-1	1	1	1
	1	1	0	-1	1	1	1
	1	1	1	0	0	0	0
1	x	x	x	0	0	0	0

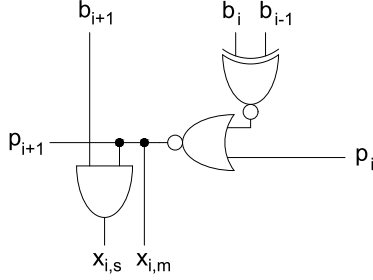


Fig. 2. Conversion of one digit from binary to CSD [32].

When the CORDIC algorithm is processed in pipeline, each micro-rotation of the conventional CORDIC according to (4) requires the calculation of one addition and one subtraction, as multiplications by powers of two have no hardware cost. Likewise, obtaining δ_i according to (5) requires one addition/subtraction per CORDIC stage. This results in a total of three additions/subtractions per CORDIC stage. As a consequence, the CORDIC algorithm is capable of calculating rotations in digital systems through a few simple operations.

C. Canonical Signed Digit Representation

The canonical signed digit (CSD) representation is a widely used approach to represent binary numbers. Whereas the binary representation considers the digits $\{0, 1\}$, in CSD the digits can take the values $\{-1, 0, 1\}$. A key property of CSD numbers is that they never have two consecutive non-zero digits. This reduces the average and maximum number of non-zero digits to $n/3 + 1/9$ and $\lceil n/2 \rceil$, respectively [32].

Table I and Fig. 2 show how to transform a binary number to CSD according to [32]. The digits of the binary number are represented as b_i , whereas the digits of the CSD number are represented as $\{x_{i,s}, x_{i,m}\}$, where $x_{i,s}$ is the sign of the i -th digit and $x_{i,m}$ is its magnitude ($0 \equiv 00, 1 \equiv 01, -1 \equiv 11$). The value p_i indicates if the previous bit was one or zero. As CSD numbers never have two consecutive '1's, if $p_i = 1$, then the current bit will be '0'. Conversely, if $p_i = 0$, then $x_{i,m} = b_i \oplus b_{i-1}$ and $x_{i,s} = x_{i,m} \cdot b_{i+1}$.

III. MOTIVATION FOR THE UD CORDIC

Table II compares the angle set that has been traditionally used in CORDIC rotators, which considers the angles in (3),

TABLE II
CORDIC CONVENTIONAL ANGLE SET VS. ANGLE SET 2^{-i}

i	Conventional Angle Set			Angle Set 2^{-i}		
	α_i (rad)	α_i (deg)	$\frac{\alpha_i}{\alpha_{i+1}}$	α_i (rad)	α_i (deg)	$\frac{\alpha_i}{\alpha_{i+1}}$
0	$\tan^{-1}(2^0)$	45.0000	1.6940	2^0	57.2958	2
1	$\tan^{-1}(2^{-1})$	26.5651	1.8926	2^{-1}	28.6479	2
2	$\tan^{-1}(2^{-2})$	14.0362	1.9700	2^{-2}	14.3239	2
3	$\tan^{-1}(2^{-3})$	7.1250	1.9923	2^{-3}	7.1620	2
4	$\tan^{-1}(2^{-4})$	3.5763	1.9981	2^{-4}	3.5810	2
5	$\tan^{-1}(2^{-5})$	1.7899	1.9995	2^{-5}	1.7905	2
6	$\tan^{-1}(2^{-6})$	0.8952	1.9999	2^{-6}	0.8952	2
7	$\tan^{-1}(2^{-7})$	0.4476	2.0000	2^{-7}	0.4476	2
8	$\tan^{-1}(2^{-8})$	0.2238	2.0000	2^{-8}	0.2238	2
9	$\tan^{-1}(2^{-9})$	0.1119	2.0000	2^{-9}	0.1119	2
10	$\tan^{-1}(2^{-10})$	0.0560	2.0000	2^{-10}	0.0560	2

to the angle set

$$\alpha_i = 2^{-i}. \quad (6)$$

For each micro-rotation, the table includes its index, i , the equation to calculate the angle in radians, the angle in degrees, and the ratio α_i/α_{i+1} .

By analyzing both alternatives, it can be observed that they differ in the first angles, whereas the last ones are basically the same. This occurs because $\phi \approx \tan(\phi)$ for small angles [33], which are the last ones in the table. By observing the ratio α_i/α_{i+1} , it is always equal to 2 for the angle set 2^{-i} . For the conventional angle set, it is 2 or almost two for the smallest angles, but not for the first angles. Based on this, the advantage of the angle set 2^{-i} is that, by normalizing the angle to the minimum angle, the δ_i values can be obtained easily from the binary representation of the angle. As the bits of a binary number take the digits $\{0, 1\}$, then the binary representation of the angle offers values of the form $\delta_i \in \{0, 1\}$, as is considered in [24]. According to this, each stage of this CORDIC either calculates a positive rotation ($\delta_i = 1$) or does not calculate any rotation ($\delta_i = 0$). However, it is generally preferable to have the option $\delta_i = \{-1, 1\}$ as in the original CORDIC because the angle is the same with a negative sign. This has the advantage that the micro-rotation stage scales the input by a constant factor. Furthermore, the micro-rotation stage is simple as the rotations by the angles α_i and $-\alpha_i$ share most of the circuit. Conversely, including the case $\delta_i = 0$, forces to create a micro-rotation by several angles that do not necessarily share the micro-rotation circuit, and also to make sure that the scaling for all the angles is the same or it is compensated. Among the solutions to this issue that exist in the literature, [23] considers variable scaling for different angles. This requires adding a circuit to compensate for the variable scaling and make it equal for all the angles. Alternatively, the approaches in [8] and [24] force the scaling of the rotator to be 1. This makes the magnitude of the output data equal to the magnitude of the inputs. However, restricting the scaling to 1 limits the set of coefficients that can be used for the micro-rotations, which makes it more difficult to obtain coefficients that calculate the micro-rotations with few shift-and-add operations.

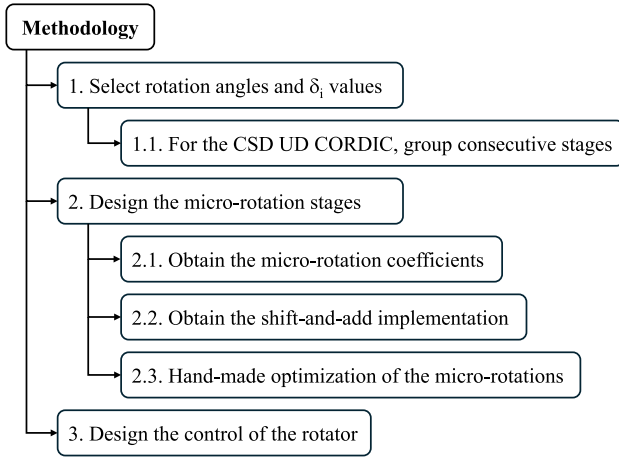


Fig. 3. Methodology for the design of the proposed UD CORDIC rotators.

To circumvent these issues, the proposed binary UD CORDIC considers $\delta_i \in \{-1, 1\}$ and allows for constant scaling at each micro-rotation stage without forcing it to be 1. The proposed CSD UD CORDIC provides further optimization based on the property that CSD numbers cannot have two consecutive non-zero digits. Thus, by transforming the binary representation of the rotation angle into a CSD number, it is guaranteed that two consecutive micro-rotations must never be calculated. This allows for merging pairs of consecutive micro-rotations into a single stage where at most one micro-rotation may be calculated.

IV. METHODOLOGY

Fig. 3 shows the methodology used to design the proposed rotators. The first step is to select the rotation angles and the δ_i values. As described in Section III, the selection of the angles $\alpha_i = 2^{-i}$ together with the use of $\delta_i \in \{-1, 1\}$ is a unique characteristic of the UD CORDIC. Additionally, for the CSD UD CORDIC, pairs of consecutive micro-rotation stages are merged, as is detailed later in Section VI.

The second step in the methodology is the design of the micro-rotation stages. This step has three parts. First, the coefficients $C + jS$ of each micro-rotation are obtained with the procedure described in [31]. It searches for coefficients with an angle close to α_i whose shift-and-add implementation uses few adders. Additionally, for the CSD UD CORDIC, where several different angles are considered, this procedure guarantees that the scaling for all the rotation angles is similar.

Once the coefficients for each micro-rotation stage are obtained, multiple-constant multiplication (MCM) and reconfigurable multiple-constant multiplication (RMCM) methods [34], [35], [36], [37], [38] are used to obtain the shift-and-add implementation of the micro-rotation stages. Additionally, the first stage of the CSD UD CORDIC, which is described in Section VI-B, has required hand-made optimizations. This is because, nowadays, RMCM algorithms can not always return an optimum solution, as is the case for this micro-rotation stage.

The third and final step in the methodology is the design of the control part of the CORDIC. Although the control part is simple, it requires a few logic gates. Sections V-D and VI-C

detail the design of the control of the binary UD CORDIC and CSD UD CORDIC, respectively.

V. BINARY UD CORDIC

Fig. 4 shows the architecture of the binary UD CORDIC. It is a pipelined design with a series of stages where trivial rotations, micro-rotations, and nano-rotations [15] are calculated. The architecture receives a continuous flow of input data $x + jy$ and provides the rotated output $X + jY$. Table III shows detailed information about all the stages of the binary UD CORDIC. The first column of the table includes the stage number. The second column shows the type of rotator at each stage. Columns three and four provide the rotation coefficients and the rotation angles in radians. Columns five, six, and seven show the number of adders that are required to calculate the micro-rotation ($\text{Add}_{\mu R}$), the adders of the control part (Add_C), and the multiplexers used in the stage (Mux), respectively. As an adder and a subtractor have similar hardware costs, throughout the paper, we refer to both as adders. Column eight shows the effective word length of the stage (WL_E), which is calculated according to equation (2). Columns nine and ten provide the angle range that each stage can receive (α_{in}) and the output range (α_{out}), respectively. Finally, the last column of the table shows the normalized scaling of the stage (R_{norm}). From the information in Fig. 4 and Table III, the following sections describe how the architecture works.

A. Trivial Rotation

The first stage of the binary UD CORDIC calculates a trivial rotation. As can be observed in Fig. 4, the trivial rotation over the input data is calculated using four multiplexers and two multiplications by -1 .

The rotation angle θ is provided in radians with 13 bits as

$$\theta(\text{rad}) \equiv \theta_{12}\theta_{11}\theta_{10} \cdot \theta_9\theta_8\theta_7\theta_6\theta_5\theta_4\theta_3\theta_2\theta_1\theta_0 \quad (7)$$

where ($\equiv$) is used to relate a number with its binary representation, and the dot in the binary representation separates the integer part and the fractional part of the binary number. This represents a number in two's complement in the range $[-4, 4)$ with a resolution of 2^{-10} radians. However, we consider that the input angle is in the range $[-\pi, \pi)$, as this range is enough to cover all the circumference.

Table IV shows how the trivial rotation is carried out. The input angle α_{in} can take a value inside different intervals between $[-\pi, 3)$ and $[3, \pi)$. Each interval is identified based on the four MSBs of θ , i.e., $\theta_{12}\theta_{11}\theta_{10}\theta_9$. For instance, any value of θ in the interval $[2.5, 3)$ has $\theta_{12}\theta_{11}\theta_{10}\theta_9 = 0101$. The goal of the trivial rotator is to set the output angle α_{out} in the range of ± 1 radian. Therefore, the rotations by 0 , $\pi/2$, π , and $-\pi/2$ radians are calculated so that this output range is guaranteed. For instance, any α_{in} in the range $[1, 2.5)$ is rotated by $\pi/2$. This sets the remaining angle α_{out} in the range $[-0.57, 0.92)$, which is inside ± 1 radians.

The control signals for the trivial rotator are t_1 and t_0 , which indicate that a rotation by π or $\pi/2$ must be calculated, respectively. In the trivial rotator of Fig. 4, it can be checked how the different angles are rotated depending on these

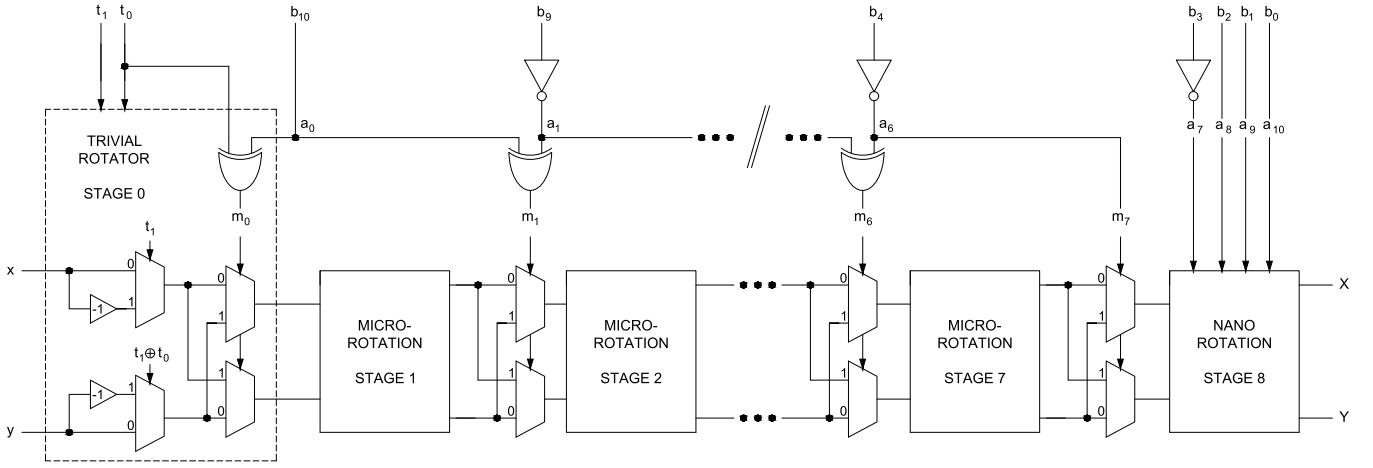


Fig. 4. Architecture of the binary UD CORDIC.

TABLE III

STAGES FOR THE BINARY UD CORDIC

Stage	Rotator type	Coefficients	Angles (rad)	Add _{μR}	Add _C	Mux.	WL _E	α _{in} (rad)	α _{out} (rad)	R _{norm}
0	Trivial rotation	$P_0 = 1$ $P_1 = j$ $P_2 = -1$ $P_3 = -j$	$\alpha_0 = 0$ $\alpha_1 = \pi/2$ $\alpha_2 = \pi$ $\alpha_3 = -\pi/2$	1	1	4	∞	$\pm\pi$	± 1	1
1	UD CORDIC	$P = 64 + j35$	$\alpha = 2^{-1}$	6	0	0(+2)	12.64	± 1	$\pm 2^{-1}$	1.1398
2	UD CORDIC	$P = 63 + j16$	$\alpha = 2^{-2}$	4	0	0(+2)	11.09	$\pm 2^{-1}$	$\pm 2^{-2}$	1.0156
3	UD CORDIC	$P = 127 + j16$	$\alpha = 2^{-3}$	4	0	0(+2)	13.09	$\pm 2^{-2}$	$\pm 2^{-3}$	1.0000
4	CORDIC	$P = 16 + j$	$\alpha = 2^{-4}$	2	0	0(+2)	15.08	$\pm 2^{-3}$	$\pm 2^{-4}$	1.0020
5	CORDIC	$P = 32 + j$	$\alpha = 2^{-5}$	2	0	0(+2)	18.08	$\pm 2^{-4}$	$\pm 2^{-5}$	1.0005
6	CORDIC	$P = 64 + j$	$\alpha = 2^{-6}$	2	0	0(+2)	21.08	$\pm 2^{-5}$	$\pm 2^{-6}$	1.0001
7	CORDIC	$P = 128 + j$	$\alpha = 2^{-7}$	2	0	0(+2)	24.08	$\pm 2^{-6}$	$\pm 2^{-7}$	1.0000
8	Nano-rotation	$P_k = 1024 + jk$ $k = 0, \dots, 8$	$\alpha_k = k \cdot 2^{-10}$	4	0	8	17.50	$\pm 2^{-7}$	$\pm 2^{-11}$	≈ 1

TABLE IV
TRIVIAL ROTATIONS

α _{in} (rad)	MSBs	Rot. Angle (rad)	α _{out} (rad)	t ₁ t ₀
[3, π)	0110	π	[-0.64, 0)	10
[2.5, 3)	0101			
[2, 2.5)	0100			
[1.5, 2)	0011	π/2	[-0.57, 0.92)	01
[1, 1.5)	0010			
[0.5, 1)	0001			
[0, 0.5)	0000	0	[-1, 1)	00
[-0.5, 0)	1111			
[-1, 0.5)	1110			
[-1.5, 1)	1101			
[-2, 1.5)	1100	-π/2	[-0.92, 0.57)	11
[-2.5, 2)	1011			
[-3, 2.5)	1010			
[-π, 3)	1001	π	[0, 0.64)	10

control signals. So far, consider that the signal that controls the two output multiplexers is t_0 . Later, in Section V-D, we will explain the reason for the XOR gate that obtains m_0 .

The number of components of the trivial rotator is reported in Table III. Both multiplications of the input data by -1 count as 0.5 adders each, so Add_{μR} = 1. The calculation of t_1 and t_0 from $\theta_{12}\theta_{11}\theta_{10}\theta_9$ according to Table IV only requires a few gates. However, in the control part, a subtractor is required to subtract the angle rotated in the trivial rotator from θ , so Add_C = 1. The result of this subtraction is the remaining angle in the range ± 1 , which is represented with the bits $b_{10} \dots b_0$ in Fig. 4. These bits are the input control signals for the micro-rotation and nano-rotation stages of the binary UD CORDIC.

B. Micro-Rotations

Stages 1 to 7 in Fig. 4 and Table III calculate micro-rotations. There are two types of them: Stages 1 to 3 are rotations specifically designed for the UD CORDIC, whereas stages 4 to 7 are the rotations of the conventional CORDIC algorithm. As the purpose of the UD CORDIC is to use angles related by a factor of two in consecutive stages, the first 3 stages need a special design to achieve this condition. This is why UD CORDIC stages are used.

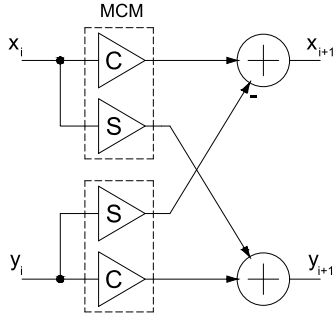
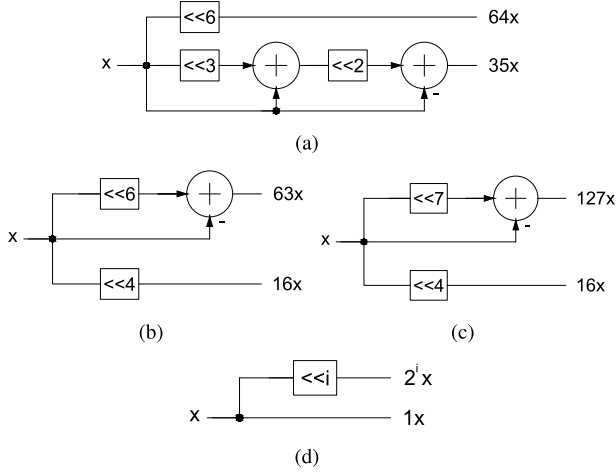


Fig. 5. Micro-rotation stage of the UD CORDIC.

Fig. 6. MCM circuits for the micro-rotation stages of the binary UD CORDIC. (a) Stage 1 by $64 + j35$. (b) Stage 2 by $63 + j16$. (c) Stage 3 by $127 + j16$. (d) Stages 4 to 7 by $2^i + j$.

Conversely, for stages 4 to 7, the conventional CORDIC already achieves a relation between consecutive angles that is almost equal to two, as was shown in Table II. Therefore, in these stages, the angles of the conventional CORDIC are used, which have the additional advantage of calculating the rotations using few adders. From another perspective, the exact rotation angle at each stage i , which is obtained as $\tan^{-1}(S_i/C_i)$ is close to $\alpha_i = 2^{-i}$ with a high accuracy measured by the effective word length provided in the eighth column of the table. This assures that the relation between consecutive angles is almost 2.

Fig. 5 shows the general structure of each micro-rotation stage. It calculates the multiplication by the coefficients $P = C + jS$ reported in the third column of Table III. First, the circuit multiplies the real and imaginary parts of the input, x and y , by C and S . These multiplications are calculated using shift-and-add operations, where each pair of multiplications by C and S is treated as an MCM problem [34], [35], [36], [37], [38].

The MCM circuits for the micro-rotations of the binary UD CORDIC are shown in Fig. 6. Fig. 6(a) shows the multiplications by $C = 64$ and $S = 35$ for stage 1. This circuit is used twice, as both x and y are multiplied by 64 and 35 according to Fig. 5. The MCM circuits for stages 2 and 3 are shown in Figs. 6(b) and 6(c), respectively. The C and S values correspond to those reported in Table III. Finally, the MCM circuit for stages 4 to 7 is shown in Fig. 6(d),

where i is the stage. These micro-rotations in stages 4 to 7 are the same micro-rotations used in the conventional CORDIC.

The number of components of each micro-rotation is reported in Table III. From Fig. 5, it can be observed that the number of adders in a micro-rotation stage is calculated as

$$\text{Add}_{\mu R} = 2 \cdot \text{Add}_{\text{MCM}} + 2, \quad (8)$$

where Add_{MCM} is the number of adders in the MCM circuit of the corresponding stage from Fig. 6. The number of adders in the control part is zero in all these stages, as will be explained in Section V-D. Additionally, each stage uses two multiplexers, which allow for calculating either a positive rotation by $C + jS$ or a negative rotation by $C - jS$. These multiplexers appear in Fig. 4. How positive and negative rotations are calculated is also clarified in Section V-D.

Regarding the effective word length (WL_E) in Table III, it is computed taking into account the distance between each angle $\alpha_i = 2^{-i}$ and the corresponding coefficient $C + jS$. Thus, the error is calculated as

$$\varepsilon = |\alpha_i + \arctan(S/C)|, \quad (9)$$

and, by applying (9) to (2), WL_E is obtained as

$$WL_E = \frac{3}{2} - \log_2 |\alpha_i + \arctan(S/C)|. \quad (10)$$

Finally, the scaling of each stage is calculated as

$$R = |P| = |C + jS|, \quad (11)$$

and the normalized scaling reported in the table is the result of dividing the scaling of the corresponding stage by the closest power of two that is smaller than R , i.e.,

$$R_{\text{norm}} = \frac{R}{2^{\lfloor \log_2 R \rfloor}}. \quad (12)$$

C. Nano-Rotation

The nano-rotation stage of the binary UD CORDIC is the same as in the CORDIC II rotator [15]. The input angle range is $\pm 2^{-7}$ rad = $\pm 0.448^\circ$ and the output angle range is $\pm 2^{-11}$ rad = $\pm 0.028^\circ$. The rotation angles in the nano-rotation are $P_k = C + jk$, where $k = 0 \dots 8$, and C is calculated by considering that $\alpha_k \approx k/C$, which leads to $C \approx k/\alpha_k = 1024$.

Table III reports the information about the nano-rotation stage. Further details about this circuit can be found in [15].

D. Control

After the trivial rotation, the remaining rotation angle in the range ± 1 is represented by the bits $b_{10} \dots b_0$ in 2's-complement, where b_{10} is an integer digit, $b_9 \dots b_0$ are fractional digits, and the angle is represented in radians. From these bits, the control block obtains the signals m_0 to m_7 that control the multiplexers between micro-rotation stages in Fig. 4.

Table V shows an example to explain the control of the UD CORDIC. For simplicity, the example considers only five bits $b_4 \dots b_0$, where the control signals m_0 to m_3 must be obtained from them. The first part of the table shows the binary

TABLE V
CONVERSION TRUTH TABLE FOR THE BINARY UD CORDIC

b_4	b_3	b_2	b_1	b_0	N	δ_0	δ_1	δ_2	δ_3	δ_{REM}	a_0	a_1	a_2	a_3	a_{REM}	m_0	m_1	m_2	m_3	m_4
0	1	1	1	1	15	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0
0	1	1	1	0	14	1	1	1	1	-1	0	0	0	0	1	0	0	0	0	0
0	1	1	0	1	13	1	1	1	-1	0	0	0	0	1	0	0	0	0	1	1
0	1	1	0	0	12	1	1	1	-1	-1	0	0	0	0	1	0	0	0	1	1
0	1	0	1	1	11	1	1	-1	1	0	0	0	1	0	0	0	0	1	1	0
0	1	0	1	0	10	1	1	-1	1	-1	0	0	1	0	1	0	0	1	1	0
0	1	0	0	1	9	1	1	-1	-1	0	0	0	1	1	0	0	0	1	0	1
0	1	0	0	0	8	1	1	-1	-1	-1	0	0	1	1	1	0	0	1	0	1
0	0	1	1	1	7	1	-1	1	1	0	0	1	0	0	0	0	1	1	0	0
0	0	1	1	0	6	1	-1	1	1	-1	0	1	0	0	1	0	1	1	0	0
0	0	1	0	1	5	1	-1	1	-1	0	0	1	0	1	0	0	1	1	1	1
0	0	1	0	0	4	1	-1	1	-1	-1	0	1	0	1	1	0	1	1	1	1
0	0	0	1	1	3	1	-1	-1	1	0	0	1	1	0	0	0	1	0	1	0
0	0	0	1	0	2	1	-1	-1	1	-1	0	1	1	0	1	0	1	0	1	0
0	0	0	0	1	1	1	-1	-1	-1	0	0	1	1	1	0	0	1	0	0	1
0	0	0	0	0	0	1	-1	-1	-1	-1	0	1	1	1	1	0	1	0	0	1
1	1	1	1	1	-1	-1	1	1	1	0	1	0	0	0	0	1	1	0	0	0
1	1	1	1	0	-2	-1	1	1	1	-1	1	0	0	0	1	1	1	0	0	0
1	1	1	0	1	-3	-1	1	1	-1	0	1	0	0	1	0	1	1	0	1	1
1	1	1	0	0	-4	-1	1	1	-1	-1	1	0	0	1	1	1	1	0	1	1
1	1	0	1	1	-5	-1	1	-1	1	0	1	0	1	0	0	1	1	1	1	0
1	1	0	1	0	-6	-1	1	-1	1	-1	1	0	1	0	1	1	1	1	1	0
1	1	0	0	1	-7	-1	1	-1	-1	0	1	0	1	1	0	1	1	1	0	1
1	1	0	0	0	-8	-1	1	-1	-1	-1	1	0	1	1	1	1	1	1	0	1
1	0	1	1	1	-9	-1	-1	1	1	0	1	1	0	0	0	1	0	1	0	0
1	0	1	1	0	-10	-1	-1	1	1	-1	1	1	0	0	1	1	0	1	0	0
1	0	1	0	1	-11	-1	-1	1	-1	0	1	1	0	1	0	1	0	1	1	1
1	0	1	0	0	-12	-1	-1	1	-1	-1	1	1	0	1	1	1	0	1	1	1
1	0	0	1	1	-13	-1	-1	-1	1	0	1	1	1	0	0	1	0	0	1	0
1	0	0	1	0	-14	-1	-1	-1	1	-1	1	1	1	0	1	1	0	0	1	0
1	0	0	0	1	-15	-1	-1	-1	-1	0	1	1	1	1	0	1	0	0	0	1
1	0	0	0	0	-16	-1	-1	-1	-1	-1	1	1	1	1	1	1	0	0	0	1

representation of all the 5-bit numbers that can be represented with $b_4 \dots b_0$. The column labeled with N , which ranges from -16 to 15 , is the decimal number corresponding to these bits according to the 2's-complement representation, i.e.,

$$N = -b_4 \cdot 2^4 + b_3 \cdot 2^3 + b_2 \cdot 2^2 + b_1 \cdot 2^1 + b_0 \cdot 2^0, \quad (13)$$

where $b_i \in \{0, 1\}$. However, we want to use $\delta_i \in \{-1, 1\}$. Therefore, we look for a new representation of N of the form

$$N = \delta_0 \cdot 2^3 + \delta_1 \cdot 2^2 + \delta_2 \cdot 2^1 + \delta_3 \cdot 2^0 + \delta_{\text{REM}} \cdot 2^0, \quad (14)$$

where $\delta_i \in \{-1, 1\}$ for $i = 0 \dots 3$ and the remaining angle $\delta_{\text{REM}} \in \{-1, 0\}$. Note that δ_{REM} must be in the range $\{-1, 0\}$ because, if $\delta_{\text{REM}} \in \{-1, 1\}$, it is not possible to generate all the values of N . By using the δ_i values, it is possible to obtain all the values of N with the advantage that rotations are either positive or negative, but not zero, except for δ_{REM} .

However, bits in a digital system are either 0 or 1. Thus, we must represent the values $\{-1, 1\}$ with a bit. According to this, the a_i values in Table V are the same as the δ_i values where 1 has been substituted by 0 and -1 has been substituted by 1. For a_{REM} , 0 in δ_{REM} remains as 0, whereas -1 is represented as 1. Therefore, when a control signal is $a_i = 1$, the rotation in the circuit is negative, and when $a_i = 0$ the rotation is positive, except for a_{REM} , where $a_{\text{REM}} = 0$ means no rotation.

In the control of the binary UD CORDIC, the b_i values are received as input and the a_i values must be obtained from

them to control the stages. By observing the values in the table it can be noted that $a_0 = b_4$, $a_1 = \bar{b}_3$, $a_2 = \bar{b}_2$, $a_3 = \bar{b}_1$, and $a_{\text{REM}} = \bar{b}_0$. Therefore, the a_i values are obtained from b_i through simple logic gates. This can also be observed in Fig. 4, where a_{REM} corresponds to a_7 .

Apart from obtaining the a_i values, the control of the binary UD CORDIC reduces the multiplexers between micro-rotation stages. The micro-rotation stages of the binary UD CORDIC only calculate positive rotations. The negative rotation by the same angle in absolute value is calculated by exchanging the real and imaginary parts of the data inputs and outputs of the stage as follows. Let us consider the input $x + jy$ and change its real and imaginary parts, which results in $y + jx$. Then, this value is rotated by $C + jS$, leading to $(yC - xS) + j(xC + yS)$. Finally, the real and imaginary parts of the output are exchanged, leading to $(xC + yS) + j(yC - xS)$. This result is the same as rotating $x + jy$ by $C - jS$. Therefore, exchanging the real and imaginary parts of the data at the input and output of the micro-rotation stage leads to a rotation by the same angle but negative.

Therefore, the CORDIC stages only need a pair of multiplexers at the input of each stage and a pair of multiplexers at the output to either exchange the real and imaginary parts or not. However, having a pair of multiplexers at the output of a micro-rotation stage and another pair of multiplexers at the input of the next micro-rotation stage is redundant, as a single pair of multiplexers is enough to exchange the values or not.

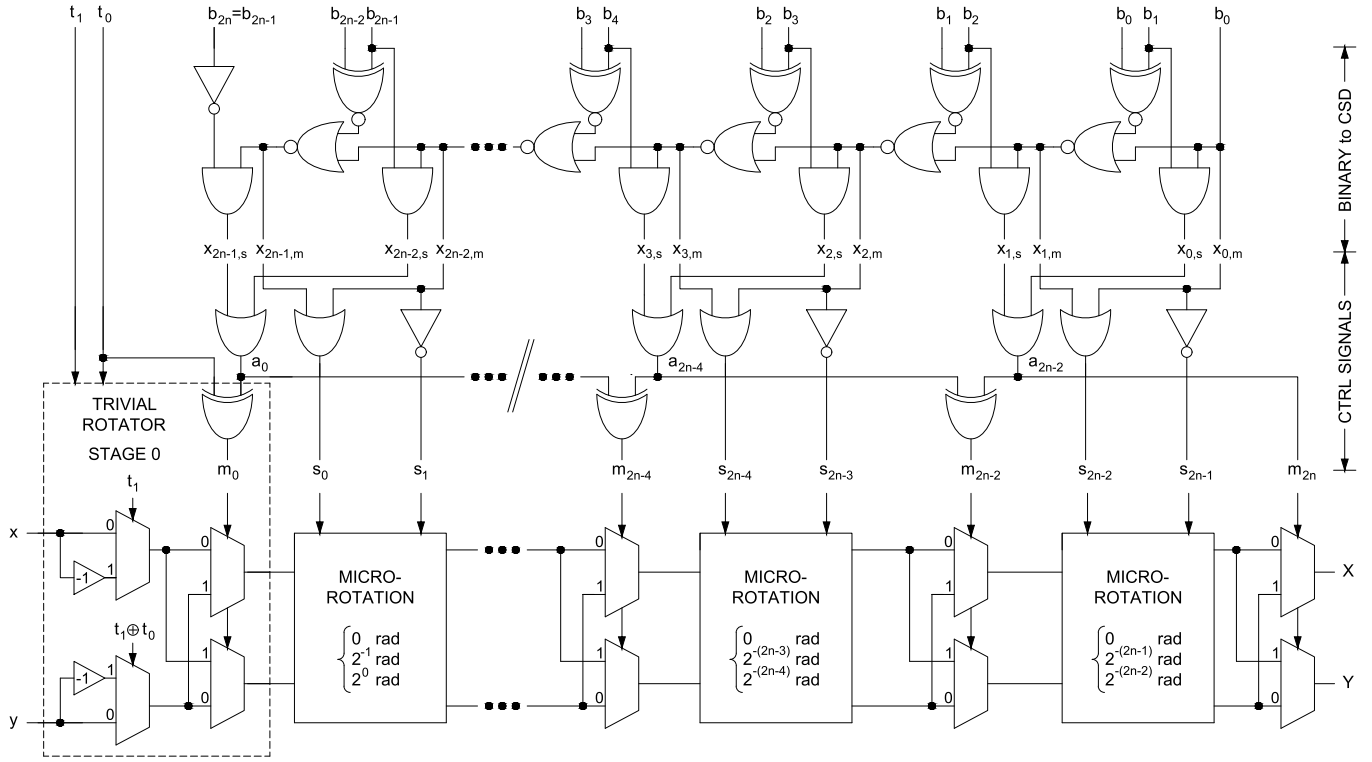


Fig. 7. Architecture of the CSD UD CORDIC.

To merge these pairs of multiplexers, we only need to have one pair of multiplexers controlled by a control signal that is equal to the XOR of the control signals of the two pairs. This idea was already used in the memoryless CORDIC [14] and is applied in the binary UD CORDIC to obtain the m_i signals. In the example of Table V the values of m_i are calculated from a_i as $m_0 = a_0$, $m_1 = a_0 \oplus a_1$, $m_2 = a_1 \oplus a_2$, $m_3 = a_2 \oplus a_3$, $m_4 = a_3$. In Fig. 4, it can be observed how the m_i values at the different stages are obtained from the a_i values through logic gates.

By observing the control part of Fig. 4, it can be noted that the control signals are obtained from b_i using a few logic gates, and no adder is required in the control part, which is one clear advantage of the binary UD CORDIC. According to this, in Table III the number of adders for the control part (Add_C) is zero for stages 1 to 8. Additionally, stages 1 to 7 include 2 multiplexers after the micro-rotation to enable either the calculation of the positive or the negative rotation. These multiplexers are indicated as (+2).

E. Scaling

The last column of Table III shows the scaling of the stages. The total scaling of the binary UD CORDIC is obtained as the product of the scaling in all the stages, leading to

$$R_{\text{Binary UD CORDIC}} = \prod_s R_{\text{norm}}(s) = 1.1606. \quad (15)$$

VI. CSD UD CORDIC

Fig. 7 shows the architecture of the CSD UD CORDIC. It shows the trivial rotation stage, a series of micro-rotation

stages, and the control part. The control part is divided into a transformation from binary to CSD and the calculation of the control signals for the micro rotation stages.

Table VI shows the information about the rotation stages in the CSD UD CORDIC with the same parameters as in Table III. Additionally, in the CSD UD CORDIC, the number of micro-rotation stages is n .

A. Trivial Rotation

The trivial rotation of the CSD UD CORDIC is the same as in the binary UD CORDIC, as described in Section V-A. The only difference is that the input angle may have a larger number of decimal bits. Specifically, the input angle is represented with $2n + 3$ bits, including 3 integer bits and $2n$ decimal bits. The remaining angle after the trivial rotation has $2n$ bits, $b_{2n-1} \dots b_0$, in the range $[-1, 1)$.

B. Micro-Rotations

Apart from the trivial rotation, the CSD UD CORDIC includes specially tailored micro-rotations, shown in Table VI. Each micro-rotation stage of the CSD UD CORDIC can rotate by a set of five angles, including positive and negative rotations. These angles are $\{-\alpha_2, -\alpha_1, \alpha_0, \alpha_1, \alpha_2\}$, where $\alpha_2 = 2\alpha_1$ and $\alpha_0 = 0$ rad. At each stage $i = 1 \dots n$, the exact values of the angles are $\alpha_0 = 0$, $\alpha_1 = 2^{-(2i-1)}$, and $\alpha_2 = 2^{-(2i-2)}$ rad.

The first micro-rotation stage considers $\alpha_0 = 0$, $\alpha_1 = 0.5$ and $\alpha_2 = 1$. The structure of the micro-rotator is the same shown in Fig. 5, where the RMCM multiplications are calculated using the circuit in Fig. 8 and correspond to the

TABLE VI
MICRO-ROTATION STAGES FOR THE CSD UD CORDIC

Stage	Rot. Type	Coefficients	Angles (rad)	Add _{uR}	Add _C	Mux.	WL _E	α_{in} (rad)	α_{out} (rad)	R_{norm}
0	Trivial rotation	$P_0 = 1$ $P_1 = j$ $P_2 = -1$ $P_3 = -j$	$\alpha_0 = 0$ $\alpha_1 = \pi/2$ $\alpha_2 = \pi$ $\alpha_3 = -\pi/2$	1	1	4	∞	$\pm\pi$	± 1	1
1	CSD UD CORDIC	$P_0 = 296$ $P_1 = 260 + j142$ $P_2 = 160 + j249$	$\alpha_0 = 0$ $\alpha_1 = 0.5$ $\alpha_2 = 1$	8	0	12(+2)	12.37	± 1	$\pm 2^{-2}$	1.1566
2	CSD UD CORDIC	$P_0 = 129$ $P_1 = 128 + j16$ $P_2 = 125 + j32$	$\alpha_0 = 0$ $\alpha_1 = 2^{-3}$ $\alpha_2 = 2^{-2}$	6	0	2(+2)	12.09	$\pm 2^{-2}$	$\pm 2^{-4}$	1.0079
3	CSD UD CORDIC	$P_0 = 32$ $P_1 = 32 + j1$ $P_2 = 32 + j2$	$\alpha_0 = 0$ $\alpha_1 = 2^{-5}$ $\alpha_2 = 2^{-4}$	2	0	2(+2)	11.50	$\pm 2^{-4}$	$\pm 2^{-6}$	1.0010
$i = 4 \dots n$	CSD UD CORDIC	$P_0 = 2^{2i-1}$ $P_1 = 2^{2i-1} + j$ $P_2 = 2^{2i-1} + j2$	$\alpha_0 = 0$ $\alpha_1 = 2^{-(2i-1)}$ $\alpha_2 = 2^{-(2i-2)}$	2	0	2(+2)	≥ 15.50	$\pm 2^{-(2i-4)}$	$\pm 2^{-(2i-2)}$	≈ 1

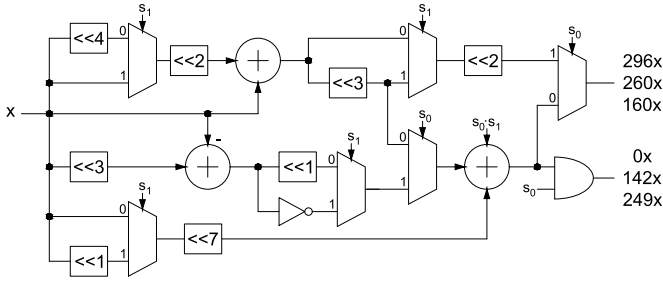


Fig. 8. Reconfigurable MCM by $296 + j0$, $260 + j142$ or $160 + j249$ for the first micro-rotation stage of the CSD UD CORDIC.

coefficients $296 + j0$, $260 + j142$, and $160 + j249$ for the angles α_0 , α_1 , and α_2 , respectively. Note that the scaling of all these coefficients is similar. In fact, the rotations by all the coefficients at any CSD UD CORDIC stage have similar scaling. We call this scaling constant even when it is not exactly the same scaling. When the scaling is the same, as in the binary UD CORDIC, the rotation error is equal to the angle difference between the angle of the coefficient and the rotation angle of this particular stage according to (9). Conversely, in the CSD UD CORDIC, there is a component of the error related to the difference in magnitude of all the coefficients in any particular stage. In this case, instead of calculating the error in terms of the angle difference between the coefficient angle and the rotation angle, the error for each rotation angle is calculated as the distance in the complex plane between each rotation coefficient and the intersection between the scaling R of the micro-rotation and the rotation angle corresponding to this coefficient. The error of the micro-rotation is then obtained as the maximum of these errors, and WL_E is then calculated according to (2). This calculation is explained in detail in [31].

The RMCM circuit in Fig. 8 is controlled by the signals s_0 and s_1 . When $s_1s_0 = 10$, the input is multiplied by $296 + j0$, and a rotation by α_0 is carried out. When $s_1s_0 = 01$, the input is multiplied by $260 + j142$, and a rotation by α_1 is carried out.

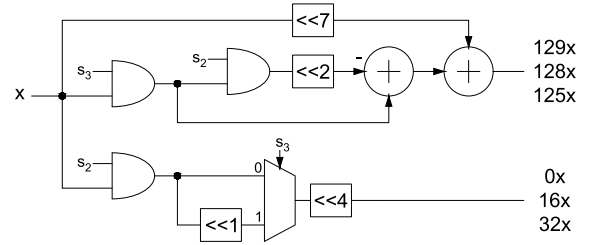


Fig. 9. Reconfigurable MCM by $129 + j0$, $128 + j16$ or $125 + j32$ for the second micro-rotation stage of the CSD UD CORDIC.

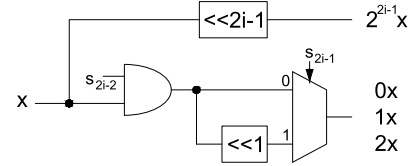


Fig. 10. Reconfigurable MCM by $2^{2i-1} + j0$, $2^{2i-1} + j1$ or $2^{2i-1} + j2$ for the i -th stage of the CSD UD CORDIC, $i = 3, \dots, n$.

Finally, when $s_1s_0 = 11$, the input is multiplied by $160 + j249$, and a rotation by α_2 is carried out.

All other micro-rotation stages follow the same approach as the first stage: They are calculated with the circuit in Fig. 5 and the multiplications by C and S are calculated using RMCM circuits. For the second stage, the rotation coefficients according to Table III are $129 + j0$ for $\alpha_0 = 0$, $128 + j16$ for $\alpha_1 = 2^{-3}$, and $125 + j32$ for $\alpha_2 = 2^{-2}$. The corresponding RMCM circuit is shown in Fig. 9. The circuit is controlled by the signals s_2 and s_3 . The multiplication by $129 + j0$ is calculated when $s_3s_2 = 10$, the multiplication by $128 + j16$ is calculated when $s_3s_2 = 01$, and the multiplication by $125 + j32$ is calculate when $s_3s_2 = 11$.

For the remaining stages, the rotation coefficients and angles are shown in Table VI, and the corresponding RMCM circuit is shown in Fig. 10.

Regarding the number of hardware components in Table VI, the first micro-rotation stage uses 8 adders, including three

TABLE VII
OBTAINING THE MICRO-ROTATION CONTROL SIGNALS OF THE CSD UD CORDIC

$x_{q+1,s}$	$x_{q+1,m}$	$x_{q,s}$	$x_{q,m}$	δ	Angle	a_{2n-2-q}	s_{2n-1-q}	s_{2n-2-q}
1	1	0	0	-2	$-\alpha_2$	1	1	1
0	0	1	1	-1	$-\alpha_1$	1	0	1
0	0	0	0	0	$\alpha_0 = 0$	0	1	0
0	0	0	1	1	α_1	0	0	1
0	1	0	0	2	α_2	0	1	1

in each RMCM circuit and two to combine the results of the RMCM circuits. Additionally, it uses 12 multiplexers. Stage 2 has two adders and one multiplexer in each RMCM circuit. By adding the two adders that combine the results of the RMCM circuits, this leads to a total of 6 adders and 2 multiplexers. Each of the remaining stages only requires two adders and two multiplexers.

Finally, it is worth noting that the CSD UD CORDIC does not include nano-rotation stages. The reason is that the CSD UD CORDIC stages for small angles are more efficient than the nano-rotation ones. This occurs because each CSD UD CORDIC stage for stages 3 or larger reduces the remaining angle by a factor of 4 using only 2 adders and 2 multiplexers, which is highly efficient.

C. Control

The control block of the CSD UD CORDIC has two parts that are marked as ‘BINARY to CSD’ and ‘CTRL SIGNALS’ at the right of Fig. 7. The first part transforms the input angle represented in binary as $b_{2n-1} \dots b_0$ to CSD. This is obtained using the CSD conversion approach described in Section II-C. The result is a CSD number where each digit can take the values $\{-1, 0, 1\}$. Each digit is represented by the pair of bits $\{x_{q,s}, x_{q,m}\}$, where $x_{q,s}$ is the sign of the q -th digit and $x_{q,m}$ is its magnitude. Thus, if the digit is 1, then $\{x_{q,s}, x_{q,m}\} = 01$, if the digit is 0, then $\{x_{q,s}, x_{q,m}\} = 00$, and if the digit is -1 , then $\{x_{q,s}, x_{q,m}\} = 11$. As a detail, note that in Fig. 7 the bit b_{2n} is needed for the CSD conversion. As $b_{2n-1} \dots b_0$ represents a number in 2’s-complement, b_{2n} is equal to b_{2n-1} due to the sign extension of the number.

The second part of the control block obtains the control signals a_q and s_q from the digits $\{x_{q,s}, x_{q,m}\}$. Table VII shows how these signals are derived. Each CSD UD CORDIC stage considers two consecutive CSD digits, $\{x_{q,s}, x_{q,m}\}$ and $\{x_{q+1,s}, x_{q+1,m}\}$. In a CSD number, in each pair of two consecutive digits, at least one of them must be zero. Therefore, the table only considers 5 cases, which are all possible combinations of these consecutive digits. These combinations appear in the first four columns of the table. The next two columns show the value of δ and the corresponding rotation angle. Note that the digit $\{x_{q+1,s}, x_{q+1,m}\}$ has weight 2 and the digit $\{x_{q,s}, x_{q,m}\}$ has weight 1, which results in the reported δ values. The values of the control signals a_{2n-2-q} , s_{2n-1-q} , and s_{2n-2-q} depend on the rotation angles. The signal a_{2n-2-q} distinguishes between positive and negative rotations, so it is only one for the negative angles. The signals s_{2n-1-q} , and s_{2n-2-q} control the micro-rotations and were described in Section VI-B. Specifically, $\{s_{2n-1-q}, s_{2n-2-q}\}$ is equal

to 10 when rotating by $\alpha_0 = 0$, to 01 when rotating by α_1 , and to 11 when rotating by α_2 . For each negative rotation, $\{s_{2n-1-q}, s_{2n-2-q}\}$ take the same value as the corresponding positive rotation.

Once Table VII has been built, it is used to obtain the control signals a_{2n-2-q} , s_{2n-1-q} , and s_{2n-2-q} from the CSD digits. This results in the following logic functions: $a_{2n-2-q} = x_{q+1,s} + x_{q,s}$, $s_{2n-1-q} = \overline{x_{q,m}}$ and $s_{2n-2-q} = x_{q+1,m} + x_{q,m}$. This leads to the logic gates shown in Fig. 7.

D. Scaling

The scaling of the CSD UD CORDIC stages is reported in the last column of Table VI. The total scaling of the CSD UD CORDIC is obtained as the product of the scaling in all the stages, leading to

$$R_{\text{CSD UD CORDIC}} = 1.1681 \quad (16)$$

VII. COMPARISON AND EXPERIMENTAL RESULTS

Table VIII compares the proposed CORDIC algorithms to recent shift-and-add rotators related to the CORDIC algorithm. The first column of the table shows the approach. Columns 2 to 6 show properties of the approach, including the type of angle set, the maximum input angle, $\alpha_{\max}$, the resolution in terms of the remaining rotation angle after the last micro-rotation stage, $\alpha_{\min}$, the range of values for δ_i , and the scaling, R . Columns 7 to 9 report the number of adders in the micro-rotations, $\text{Add}_{\mu R}$, the number of adders used for control, Add_C , and the total number of adders. Finally, column 10 shows the latency of the rotator in terms of rotation stages.

For the proposed CORDIC algorithms, Table VIII includes four rotators: the binary UD CORDIC and three alternatives for the CSD UD CORDIC, for $n = 6, 7$, and 8.

By observing the results in the table, it can be noted that recent CORDIC-based rotators tend to explore new angle sets. Among all the approaches in the table, only [25] uses the classical angle set of $\tan^{-1}(2^{-i})$. The rest of the works explore angle sets with multiple different angles or use the angle set 2^{-i} as in the proposed approach.

Most works in Table VIII consider the full circumference range of $\pm 180^\circ$ for the input rotation angle. Only [25], [26] assume that the input rotation angle is in a range of $\pm 90^\circ$.

Regarding the values for δ_i , there is a variety of alternatives. The work in [25] and the proposed binary UD CORDIC consider $\delta_i \in \{-1, 1\}$. This means that the positive and negative values of the same angle are rotated. This has the advantage that the micro-rotation stages only use one angle and its symmetric, where the symmetric one may be

TABLE VIII
COMPARISON OF CORDIC APPROACHES

Approach	Properties					Adders			Latency
	Angle Set	$\alpha_{\max}$	$\alpha_{\min}$	δ_i	R	Add _{μR}	Add _C	Total	(rot. st.)
Mixed-Radix (2021) [25]	$\tan^{-1}(2^{-i})$	$\pm 90^\circ$	$\pm 0.007^\circ$	$\{-1, 1\}$	1.6425	26	6	32	7
Mixed-Radix (2023) [26]	Multiple	$\pm 90^\circ$	$\pm 0.0035^\circ$	Multiple	1	38	4	42	6
CORDIC II (2016) [15]	Multiple	$\pm 180^\circ$	$\pm 0.056^\circ$	Multiple	1.575	16	9	25	6
CORDIC II bis (2016) [15]	Multiple	$\pm 180^\circ$	$\pm 0.028^\circ$	Multiple	1.575	18	10	28	7
Augmented Taylor Series (2019) [8]	2^{-i}	$\pm 180^\circ$	$\pm 0.056^\circ$	$\{-1, 0, 1\}$	1	29	1	30	5
Base-2 Exponent Expan. (2021) [24]	2^{-i}	$\pm 180^\circ$	$\pm 0.0035^\circ$	$\{0, 1\}$	1	NP	1	NP	5
Proposed Binary UD CORDIC	2^{-i}	$\pm 180^\circ$	$\pm 0.028^\circ$	$\{-1, 1\}$	1.1606	27	1	28	9
Proposed CSD UD CORDIC ($n = 6$)	2^{-i}	$\pm 180^\circ$	$\pm 0.056^\circ$	$\{-2, -1, 0, 1, 2\}$	1.1681	23	1	24	7
Proposed CSD UD CORDIC ($n = 7$)	2^{-i}	$\pm 180^\circ$	$\pm 0.0014^\circ$	$\{-2, -1, 0, 1, 2\}$	1.1681	25	1	26	8
Proposed CSD UD CORDIC ($n = 8$)	2^{-i}	$\pm 180^\circ$	$\pm 0.0035^\circ$	$\{-2, -1, 0, 1, 2\}$	1.1681	27	1	28	9

NP = Not provided and cannot be deduced from the paper.

calculated by exchanging the real and imaginary parts of the inputs and outputs of the stage, as explained in Section V-D. Additionally, using $\delta_i \in \{-1, 1\}$ has the advantage that the scaling is constant at each micro-rotation stage and, therefore, the scaling at the output of the rotator is constant for any input angle. Conversely, other approaches that use $\delta_i \in \{0, 1\}$ such as [24] or $\delta_i \in \{-1, 0, 1\}$ such as [8] include more than one non-symmetric angles per micro-rotation stage and must make the scaling of all the angles in each micro-rotation stage similar to make sure that all the scaling at the output of the rotator is constant for any input rotation angle. Finally, the proposed CSD UD CORDIC uses $\delta_i \in \{-2, -1, 0, 1, 2\}$, which results from combining pairs of consecutive micro-rotation stages into one stage. This halves the number of micro-rotation stages.

Regarding scaling, some approaches force it to be equal to one [8], [24], [26]. This is advantageous from the point of view that the output data has the same magnitude as the input data. The other approaches have constant scaling as the original CORDIC [1]. This is often valid, as all input data are scaled by the same value. Furthermore, this scaling can easily be compensated with a constant multiplication by the inverse of the scaling, which generally requires a few adders.

By comparing the number of adders, we can observe that rotators using the conventional angle set or multiple angle sets require several adders for the control block. Conversely, rotators based on the angle set 2^{-i} only require 1 adder for the control. This removes most of the logic in this part of the circuit. This adder comes from the two multiplications by -1 in the trivial rotator, each of which counts as 0.5 adders.

To compare the total number of adders, we take into account the resolution of the rotators in terms of $\alpha_{\min}$. For $\alpha_{\min} = \pm 0.028^\circ$, both the proposed binary UD CORDIC and the CORDIC II bis algorithm [15] require 28 adders. However, by comparing Table III of the proposed binary UD CORDIC with Table II in [15], it can be observed that the binary UD CORDIC reduces the number of multiplexers from 31 in the CORDIC II to 26.

For the resolution $\alpha_{\min} = \pm 0.056^\circ$, the CSD UD CORDIC is the most efficient solution in terms of adders, with a total of

only 24. This improves the 25 adders of the CORDIC II [15] and the 30 adders of [8]. For the resolution $\alpha_{\min} = \pm 0.0035^\circ$, the proposed CSD UD CORDIC requires only 28 adders, compared to 42 in [26]. Additionally, compared to [25], the proposed CSD UD CORDIC improves the number of adders from 32 to 28 and the resolution from $\pm 0.007^\circ$ to $\pm 0.0035^\circ$.

Finally, the latency in terms of rotation stages of the binary and CSD UD CORDIC algorithms is slightly higher than the latency in other approaches with the same resolution, as they include 2 or 3 additional rotation stages. However, this metric is merely informative and should not be taken as a measurement of the actual latency of the circuit because some stages may require carrying out more mathematical calculations than others and because the actual latency depends on the pipelining of the stages using registers, where more complex stages may require deeper pipelining than others.

Regarding experimental results, the proposed binary UD CORDIC and three variants of the CSD UD CORDIC for $n = 6, 7, 8$ have been implemented on a Virtex XC7VX330TFFG1157-1 FPGA. Table IX compares the experimental results of the proposed approach to previous CORDIC-based algorithms on the same FPGA. The first column of the table shows the approach. The second column includes the FPGA device. Column three shows the word length (WL), which is equal to 16 bits for all the approaches, i.e., 16 bits for the real part of the input and 16 bits for its imaginary part. Column four shows the resolution, $\alpha_{\min}$. The rest of the columns show the experimental results in terms of area (LUTs, FFs, and slices), clock frequency (f_{CLK}), latency in cycles and nanoseconds, and power consumption in mW. For the proposed CSD UD CORDIC implementations, the latency is $n + 6$ clock cycles. Finally, the last column of the table shows the normalized power consumption for a frequency of 400 MHz.

To compare the CORDIC implementations, we consider approaches with the same or similar resolution. Compared to [15], the binary UD CORDIC achieves the same resolution and reduces the number of FFs by 25% and the power consumption by 11% for the same clock frequency

TABLE IX

COMPARISON OF CORDIC APPROACHES IMPLEMENTED ON FPGAS

Approach	FPGA Device	Parameters		Area			f_{CLK} (MHz)	Latency		Power (mW)	
		WL	α_{min}	LUTs	FFs	Slices		cycles	ns	@ f_{CLK}	@400 MHz
CORDIC II bis (2016)* [15]	Virtex 7	16	$\pm 0.028^\circ$	573	527	189	350	9	26	81	92
Augmented Taylor Series (2019) [†] [8]	Virtex 7	16	$\pm 0.056^\circ$	1106	274	326	143	5	35	95	265
Mixed-Radix (2021) [†] [25]	Virtex 7	16	$\pm 0.007^\circ$	753	395	194	253	8	31	78	124
Mixed-Radix (2023) [†] [26]	Virtex 7	16	$\pm 0.0035^\circ$	1093	306	316	200	7	35	106	212
Proposed Binary UD CORDIC	Virtex 7	16	$\pm 0.028^\circ$	569	392	187	350	10	29	72	82
Proposed CSD UD CORDIC ($n = 6$)	Virtex 7	16	$\pm 0.056^\circ$	620	517	185	400	12	30	101	101
Proposed CSD UD CORDIC ($n = 7$)	Virtex 7	16	$\pm 0.014^\circ$	655	549	209	400	13	33	108	108
Proposed CSD UD CORDIC ($n = 8$)	Virtex 7	16	$\pm 0.0035^\circ$	700	591	211	400	14	35	112	112

*: Results of our previous work [15] on a Virtex 7 XC7VX330TFFG1157-1 FPGA.

[†]: Results on a Virtex 7 XC7VX330TFFG1157-1 FPGA provided by the authors of the paper.

TABLE X

COMPARISON OF CORDIC APPROACHES IMPLEMENTED ON ASICs

Approach	Tech. (nm)	Voltage (V)	Parameters		Area (μm^2)	Equiv. NAND2	f_{CLK} (MHz)	Latency		Power (mW)	
			WL	α_{min}				cyc.	ns	@ f_{CLK}	@800 MHz
CORDIC II bis (2016)* [15]	40	1.1	16	$\pm 0.028^\circ$	4496	6373	800	9	11.2	4.85	4.85
Enhanced Scaling-Free (2010) [6]	65	-	16	$\pm 0.0035^\circ$	13689	-	700	9	12.8	7.89	9.02
Prop. Binary UD CORDIC	40	1.1	16	$\pm 0.028^\circ$	3887	5510	800	10	12.5	2.15	2.15
Prop. CSD UD CORDIC ($n = 6$)	40	1.1	16	$\pm 0.056^\circ$	4808	6815	800	12	15.0	5.53	5.53
Prop. CSD UD CORDIC ($n = 7$)	40	1.1	16	$\pm 0.014^\circ$	5364	7602	800	13	16.2	6.28	6.28
Prop. CSD UD CORDIC ($n = 8$)	40	1.1	16	$\pm 0.0035^\circ$	5887	8343	800	14	17.5	7.02	7.02

*: Results of our previous work [15] on 40 nm ASIC technology.

of 350 MHz. It also slightly reduces the number of slices, whereas its calculation takes one clock cycle more.

Compared to [8], the proposed CSD UD CORDIC for $n = 6$ achieves the same resolution and reports 44% fewer LUTs and a larger number of FFs due to the pipelining. This results in 43% fewer slices from 326 in [8] down to 185 in the CSD UD CORDIC for $n = 6$. The proposed architecture also increases the clock frequency by a factor of 2.8 and reduces the normalized power consumption by 62% from 265 mW to 101 mW. Finally, [8] has lower latency in clock cycles, whereas the proposed architecture achieves lower latency in nanoseconds thanks to its higher clock frequency.

Compared to [25], the proposed CSD UD CORDIC for $n = 8$ obtains the closest resolution, halving that in [25]. The number of LUTs in the proposed architecture is 7% smaller and the number of FFs is 33% higher, leading to 194 slices in [25] and 211 in the proposed rotator. However, as reported in Table VIII, [25] does not include the trivial rotation to cover a range of $\pm 180^\circ$. The clock frequency of the proposed rotator is 58% higher and the normalized power consumption 10% lower, whereas the latency of [25] in ns is 11% lower due to a smaller number of pipeline stages.

Compared to [26], the proposed CSD UD CORDIC for $n = 8$ achieves the same resolution while saving 33% of slices. It also doubles the clock frequency, reduces the normalized power by 47%, and achieves the same latency in nanoseconds.

In general, the proposed UD CORDIC implementations save area, increase clock frequency, and reduce power consumption

with respect to previous approaches, whereas they have higher latency due to a larger number of pipeline stages.

By comparing the proposed implementations, the binary UD CORDIC halves the resolution with respect to the CSD UD CORDIC for $n = 6$. At the same time, it requires 8% fewer LUTs and 24% fewer FFs, despite a small increase in slices. It also reduces the latency in clock cycles and nanoseconds despite a smaller clock frequency, and reduces the normalized power by 19%. Thus, the binary UD CORDIC is an efficient approach for a resolution of $\pm 0.028^\circ$.

Conversely, for higher resolution, the CSD UD CORDIC provides a very efficient solution. By comparing the cases for $n = 6, 7$, and 8, it can be observed that the differences in area, latency, and power consumption are relatively small compared to the significant improvement in resolution. This is because adding an extra micro-rotation stage to the CSD UD CORDIC improves the resolution by a factor of 4 at the cost of adding only 2 adders. In conventional CORDIC rotators, adding one extra micro-rotation stage requires 3 adders, 2 of them for the data and 1 for the calculation of the remaining angle. Thus, a conventional CORDIC requires 6 adders to reduce the resolution by a factor of 4. This cost is 3 times smaller in the CSD UD CORDIC, as it only requires 2 adders. This makes the CSD UD CORDIC an outstanding solution to reach high resolution at low hardware cost.

Table X shows the experimental results of the proposed UD CORDIC algorithms implemented on 40 nm ASIC technology and compares them to previous works in the literature.

The proposed implementations use a voltage of 1.1 V and a 16-bit word length. All of them reach a clock frequency of 800 MHz, and they differ in area, NAND2 equivalent gate count, latency, and power consumption.

Compared to the CORDIC II bis [15] implemented on 40 nm technology, the proposed binary UD CORDIC achieves the same resolution and clock frequency with a reduction of 14% in area from 4496 μm^2 to 3887 μm^2 and savings of 55% in power consumption from 4.85 mW to 2.15 mW. Note that these improvements on ASIC are more remarkable than the improvements reported for FPGAs in Table IX, where both approaches require a similar number of slices. Thus, the larger number of FFs shown in the FPGA comparison for the CORDIC II bis results in a noticeably larger area and significantly higher power consumption on ASICs.

By comparing [6] to the proposed CSD UD CORDIC for $n = 8$, which has the same resolution, the CSD UD CORDIC for $n = 8$ increases the clock frequency, reduces the area, and lowers the power consumption at the cost of higher latency. However, as both approaches use different technology and the voltage is not reported in [6], the comparison is not evident and part of the improvement must be due to the difference in technology.

Among the proposed designs, the binary UD CORDIC achieves the smallest values in area, equivalent NAND2 gates, latency, and power consumption. However, the CSD UD CORDIC can achieve higher resolution. It is worth noting that the binary UD CORDIC reduces the area of the CSD UD CORDIC for $n = 6$ by 20% and the power consumption by 61%. Regarding CSD UD CORDIC implementations for $n = 6, 7$, and 8, a progressive increase in area, latency, and power consumption can be observed when increasing the number of stages. This is analogous to the tendency observed in the FPGA results reported in Table IX.

As a result, the proposed UD CORDIC algorithms not only require the smallest number of adders for a given resolution among pipelined CORDIC-based rotators but also result in efficient implementations on FPGAs and ASICs. This makes them an excellent solution to calculate rotations in digital systems.

VIII. CONCLUSION

This paper has presented a new algorithm to calculate rotations called UD CORDIC in two variants: binary UD CORDIC and CSD UD CORDIC. These variants simplify the control part of the CORDIC, as they obtain the control signals for the micro-rotation stages by simple transformations of the binary representation of the rotation angle. Additionally, they simplify the micro-rotation stages thanks to using $\delta_i \in \{-1, 1\}$ in the binary UD CORDIC and to merging consecutive micro-rotation stages in the CSD UD CORDIC. This results in highly efficient rotators in terms of hardware components, compared to previous pipelined CORDIC-based rotators. Additionally, whereas conventional approaches require 6 adders to reduce the CORDIC resolution by a factor of 4, the CSD UD CORDIC achieves this reduction by only adding a micro-rotation stage with only 2 adders. This makes it an outstanding solution to achieve

high-resolution rotators. To verify the proposed CORDIC algorithms, experimental results on FPGA and ASIC have been obtained, showing improvements with respect to previous approaches in multiple figures of merit, mostly area, clock frequency, and power consumption.

ACKNOWLEDGMENT

The authors want to thank Wadkar et al. [8] and Changela et al. [25], [26] for providing experimental results of their designs for a Virtex 7 FPGA. In particular, they want to thank Dr. Ankur Changela, Prof. Bishnu Prasad Das, and his Ph.D. student Anu Verma for generating and sharing these results. They also want to thank the anonymous reviewers for their valuable and constructive comments. Their key insights have been relevant for improving the quality of this article.

REFERENCES

- [1] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Trans. Electron. Comput.*, vol. 8, no. 3, pp. 330–334, Sep. 1959.
- [2] C.-S. Wu, A.-Y. Wu, and C.-H. Lin, "A high-performance/low-latency vector rotational CORDIC architecture based on extended elementary angle set and trellis-based searching schemes," *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process.*, vol. 50, no. 9, pp. 589–601, Sep. 2003.
- [3] R. Shukla and K. C. Ray, "Low latency hybrid CORDIC algorithm," *IEEE Trans. Comput.*, vol. 63, no. 12, pp. 3066–3078, Dec. 2014.
- [4] C.-S. Wu and A.-Y. Wu, "Modified vector rotational CORDIC (MVR-CORDIC) algorithm and architecture," *IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process.*, vol. 48, no. 6, pp. 548–561, Jun. 2001.
- [5] S. Aggarwal, P. K. Meher, and K. Khare, "Area-time efficient scaling-free CORDIC using generalized micro-rotation selection," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 20, no. 8, pp. 1542–1546, Aug. 2012.
- [6] F. J. Jaime, M. A. Sanchez, J. Hormigo, J. Villalba, and E. L. Zapata, "Enhanced scaling-free CORDIC," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 57, no. 7, pp. 1654–1662, Jul. 2010.
- [7] Y. Liu, L. Fan, and T. Ma, "A modified CORDIC FPGA implementation for wave generation," *Circuits, Syst., Signal Process.*, vol. 33, no. 1, pp. 321–329, Jan. 2014.
- [8] S. S. Wadkar, B. P. Das, and P. K. Meher, "Low latency scaling-free pipeline CORDIC architecture using augmented Taylor series," in *Proc. IEEE Int. Symp. Smart Electron. Syst. (iSES)*, Rourkela, India, Dec. 2019, pp. 312–315.
- [9] C.-Y. Yu, S.-G. Chen, and J.-C. Chih, "Efficient CORDIC designs for multi-mode OFDM FFT," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, vol. 3, May 2006, pp. 1036–1039.
- [10] A. Chakraborty and A. Banerjee, "CORDIC-based high-speed VLSI architecture of transform model estimation for real-time imaging," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 1, pp. 215–226, Jan. 2021.
- [11] N. Takagi, T. Asada, and S. Yajima, "Redundant CORDIC methods with a constant scale factor for sine and cosine computation," *IEEE Trans. Comput.*, vol. 40, no. 9, pp. 989–995, Sep. 1991.
- [12] A. Changela, M. Zaveri, and A. Lakhlani, "ASIC implementation of high performance Radix-8 CORDIC algorithm," in *Proc. Int. Conf. Adv. Comput., Commun. Informat. (ICACCI)*, Sep. 2018, pp. 699–705.
- [13] C.-Y. Chen and C.-Y. Lin, "High-resolution architecture for CORDIC algorithm realization," in *Proc. Int. Conf. Commun., Circuits Syst.*, vol. 1, Jun. 2006, pp. 579–582.
- [14] M. Garrido and J. Grajal, "Efficient memoryless CORDIC for FFT computation," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, vol. 2, Apr. 2007, pp. 113–116.
- [15] M. Garrido, P. Källström, M. Kumm, and O. Gustafsson, "CORDIC II: A new improved CORDIC algorithm," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 63, no. 2, pp. 186–190, Feb. 2016.
- [16] P. Källström, M. Garrido, and O. Gustafsson, "Low-complexity rotators for the FFT using base-3 signed stages," in *Proc. IEEE Asia Pacific Conf. Circuits Syst.*, Dec. 2012, pp. 519–522.
- [17] R. Andraka, "A survey of CORDIC algorithms for FPGA based computers," in *Proc. ACM/SIGDA Int. Symp. FPGAs*, Feb. 1998, pp. 191–200.

- [18] P. K. Meher, J. Valls, T.-B. Juang, K. Sridharan, and K. Maharatna, "50 years of CORDIC: Algorithms, architectures, and applications," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 56, no. 9, pp. 1893–1907, Sep. 2009.
- [19] A. Changela, M. Zaveri, and D. Verma, "A comparative study on CORDIC algorithms and applications," *J. Circuits, Syst. Comput.*, vol. 32, no. 5, pp. 1–47, Mar. 2023.
- [20] Y.-N. Chang and K. K. Parhi, "An efficient pipelined FFT architecture," *IEEE Trans. Circuits Syst. II: Analog Digit. Signal Process.*, vol. 50, no. 6, pp. 322–325, Jun. 2003.
- [21] A. Banerjee, A. S. Dhar, and S. Banerjee, "FPGA realization of a CORDIC based FFT processor for biomedical signal processing," *Microprocessors Microsyst.*, vol. 25, no. 3, pp. 131–142, May 2001.
- [22] A. Banerjee and A. S. Dhar, "Pipelined VLSI architecture using CORDIC for transform domain equalizer," *J. Signal Process. Syst.*, vol. 70, no. 1, pp. 39–48, Jan. 2013.
- [23] A. Chakraborty and A. Banerjee, "Low latency semi-iterative CORDIC algorithm using normalized angle recoding and its VLSI implementation," in *Proc. Int. Conf. Commun. Signal Process. (ICCCSP)*, Apr. 2019, pp. 0913–0920.
- [24] Y.-S. Wang, Y.-Q. Xue, H. You, and S.-S. Qiao, "Area-energy efficient CORDICs using new elementary-angle-set and base-2 exponent expansions scheme," *IEICE Electron. Exp.*, vol. 18, no. 2, Jan. 2021, Art. no. 20200409.
- [25] A. Changela, M. Zaveri, and D. Verma, "Mixed-radix, virtually scaling-free CORDIC algorithm based rotator for DSP applications," *Integration*, vol. 78, pp. 70–83, May 2021.
- [26] A. Changela, M. Zaveri, and Y. Kumar, "A new angle set-based absolute scaling-free reconfigurable cordic algorithm," *Circuits, Syst., Signal Process.*, vol. 42, no. 12, pp. 7404–7432, Dec. 2023.
- [27] H. Mahdavi and S. Timarchi, "Improving architectures of binary signed-digit CORDIC with generic/specific initial angles," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 7, pp. 2297–2304, Jul. 2020.
- [28] J. Deng, T. N. V. Kaushik, P. R. S. Prasanna, M.-S. Yang, and P. V. S. Prasad, "Study of iterations reduction for CORDIC computation with more accuracy," in *Proc. Int. Conf. Consum. Electron.-Taiwan*, Jul. 2023, pp. 311–312.
- [29] M. Qin, T. Liu, B. Hou, Y. Gao, Y. Yao, and H. Sun, "A low-latency RDP-CORDIC algorithm for real-time signal processing of edge computing devices in smart grid cyber-physical systems," *Sensors*, vol. 22, no. 19, p. 7489, Oct. 2022.
- [30] P. Paz and M. Garrido, "CORDIC-based computation of arcsine and arccosine functions on FPGA," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 9, pp. 3684–3688, Sep. 2023.
- [31] M. Garrido, F. Qureshi, and O. Gustafsson, "Low-complexity multiplierless constant rotators based on combined coefficient selection and shift-and-add implementation (CCSSI)," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 61, no. 7, pp. 2002–2012, Jul. 2014.
- [32] M. Faust, O. Gustafsson, and C.-H. Chang, "Fast and VLSI efficient binary-to-CSD encoder using bypass signal," *Electron. Lett.*, vol. 47, no. 1, pp. 18–20, 2011.
- [33] R. Larson, R. P. Hostetler, and B. H. Edwards, *Calculus of a Single Variable: Early Transcendental Functions*. Boston, MA, USA: Cengage, 2006.
- [34] M. Kumm, "Optimal constant multiplication using integer linear programming," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 65, no. 5, pp. 567–571, May 2018.
- [35] F. D. Dinechin and M. Kumm, *Application-Specific Arithmetic—Computing Just Right for the Reconfigurable Computer and the Dark Silicon Era*. Cham, Switzerland: Springer, 2024.
- [36] R. Garcia and A. Volkova, "Toward the multiple constant multiplication at minimal hardware cost," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 5, pp. 1976–1988, May 2023.
- [37] N. Fiege, M. Kumm, and P. Zipf, "Bit-level optimized constant multiplication using Boolean satisfiability," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 1, pp. 249–261, Jan. 2024.
- [38] O. Gustafsson, "A difference-based adder graph heuristic for multiple constant multiplication problems," in *Proc. IEEE Int. Symp. Circuits Syst.*, May 2007, pp. 1097–1100.



Mario Garrido (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in telecommunications engineering from the Universidad Politécnica de Madrid (UPM), Spain, in 2004 and 2009, respectively.

He was a Post-Doctoral Researcher with the Department of Electrical Engineering, Linköping University, Sweden, in 2010. From 2012 to 2019, he was an Associate Professor with the Department of Electrical Engineering, Linköping University.

In 2019, he returned to UPM with a Ramón y Cajal Research Fellowship and since 2024, he has been an Associate Professor with the Department of Electronic Engineering, UPM. So far, he has been the author of more than 50 scientific publications. His research interests include optimized hardware design for signal-processing applications, design of hardware architectures for the fast Fourier transform (FFT), circuits for data management, the CORDIC algorithm, neural networks, and circuits to calculate statistical and mathematical operations. His research covers high-performance circuits for real-time computation and designs for small area and low power consumption. He appeared in the "World's Top 2% Scientists List" elaborated by Stanford University in 2022, 2023, and 2024.



Daniel Medina received the B.S. degree in telecommunications engineering from the Universidad de Las Palmas de Gran Canaria in 2019 and the M.S. degree in electronic systems engineering from the Universidad Politécnica de Madrid, Spain, in 2020. He was at the Universidad Politécnica de Madrid, he worked on the topic of this paper. In 2019, he was a Research Intern in wireless technologies for the IoT devices at the Institute of Applied Microelectronics (IUMA). Since 2023, he has been an FPGA Engineer at AeroLaser System S.L. His interests include

hardware design of complex architectures aimed at achieving fast and efficient designs for real-time computation.



Pedro Paz received the B.S. degree in telecommunications engineering and the M.S. degree in electronic systems engineering from the Universidad Politécnica de Madrid, Spain, in 2019 and 2020, respectively, where he is currently pursuing the Ph.D. degree. Since 2020, he has been an FPGA Engineer at Indra Sistemas. His research interests include the design of hardware architectures for signal-processing algorithms, such as the fast Fourier transform (FFT) and the CORDIC algorithm.



Marisa López-Vallejo (Senior Member, IEEE) received the M.S. and Ph.D. degrees from the Universidad Politécnica de Madrid, Madrid, Spain, in 1993 and 1999, respectively.

She was a member of the Technical Staff at Lucent Technologies, Bell Laboratories, Murray Hill, NJ, USA. From 2015 to 2016, she was a Visiting Professor at the Microsystems Technology Laboratory, Massachusetts Institute of Technology, USA. Since 2016, she has been a Full Professor at the Department of Electronic Engineering, Universidad Politécnica de Madrid. Her research interests include low-power, radiation- and PVT-aware design, computer-aided diagnostic methods and tools, application-specific high-performance programmable architectures, and memristive memories. In the last decade, her research has focused on efficient architectures to support brain-inspired computing. She has coordinated many national and international projects in these areas. She has supervised 12 Ph.D. theses and has published more than 100 papers in journals and conferences related to these fields.